Bicol University Polangui, Albay
Web System and Technology
2nd Semester S.Y 2025-2026

IT 112 WEB SYSTEMS AND TECHNOLOGIES

BU TRANSAKTO

Week 12: Phase 2 of Final Project

GROUP NAME: KMJ's

MEMBERS:

BELLEN, JUVY

HERMO, MA. MERA

INTIA, KIMBERLY ANNE

REALUBIT, JANET

SEPTO, JESSECA

COURSE & BLOCK:

BSIT-2A

I. OVERVIEW OF BACKEND SETUP

The backend for BU Transakto was built using Node.js and Express.js to create a RESTful API that handles all server-side operations for the Registrar Document Request System. The setup began by initializing a new Node.js project and installing essential packages including express, mongoose, cors, dotenv, bcryptjs, jsonwebtoken, and nodemon. The folder structure was organized with separate directories for config, models, controllers, routes, and middleware to ensure modularity and scalability. The server runs on port 5000 and connects to MongoDB Atlas for cloud database persistence using Mongoose ODM. Authentication is implemented using JWT tokens with bcryptjs for password hashing, and role-based middleware restricts access between student and staff endpoints. All API endpoints were tested using Thunder Client and are ready for frontend integration.

Key Features Implemented:

- Express.js server with modular architecture
- MongoDB connection using Mongoose (Atlas cloud)
- User and Request database models
- JWT authentication with register/login endpoints
- Role-based access control (student/staff)
- RESTful API endpoints
- CRUD operations for document requests
- Staff dashboard statistics endpoints

II. CODE SNIPPETS

server.js

```
JS server.js ×
backend > JS server.js > ...
const dns = require('dns');
dns.setServers(['8.8.8.8', '1.1.1.1']);

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const app = express();

// Connect to MongoDB Atlas
connectDB();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
const authRoutes = require('./routes/authRoutes');
const userRoutes = require('./routes/userRoutes');
const requestRoutes = require('./routes/requestRoutes');
const staffRoutes = require('./routes/staffRoutes');

app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/requests', requestRoutes);
app.use('/api/staff', staffRoutes);
```

```
// Test route
app.get('/', (req, res) => {
  res.json({ message: 'BU Transakto API is running!' });
});

// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`\n🚀 Server running on port ${PORT}`);
  console.log('\n✅ Available endpoints:');
  console.log('   POST   /api/auth/register');
  console.log('   POST   /api/auth/login');
  console.log('   GET    /api/auth/me');
  console.log('   POST   /api/requests');
  console.log('   GET    /api/requests/myrequests');
  console.log('   GET    /api/requests/all (Staff only)');
  console.log('   PUT    /api/requests/:id/status (Staff only)');
  console.log('   GET    /api/staff/stats (Staff only)');
  console.log('   GET    /api/staff/students (Staff only)');
  console.log('   GET    /api/users (Staff only)');
});
```

Models

Request.js

```
Request.js
backend > models > Request.js > ...
const mongoose = require('mongoose');

const requestSchema = new mongoose.Schema({
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  studentName: { type: String, required: true },
  studentNumber: { type: String },
  documentType: {
    type: String,
    enum: ['Transcript of Records', 'Certificate of Enrollment', 'Good Moral Certificate', 'Diploma', 'Copy of Grades
    required: true
  },
  purpose: { type: String },
  notes: { type: String },
  status: {
    type: String,
    enum: ['pending', 'processing', 'completed', 'rejected'],
    default: 'pending'
  },
  dateRequested: { type: Date, default: Date.now },
  dateProcessed: { type: Date },
  processedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});

module.exports = mongoose.model('Request', requestSchema);
```

User.js

```
User.js
C:\Users\Geline\Desktop\FINAL-PROJECT-KMJs-BSIT2A\backend\models\User.js (preview)
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['student', 'staff'], required: true },
  studentId: { type: String },
  course: { type: String },
  yearLevel: { type: String },
  contactNumber: { type: String },
  address: { type: String },
  createdAt: { type: Date, default: Date.now }
});

// Hash password
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password
userSchema.methods.comparePassword = async function(enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

Routes

authRoutes.js

```
JS authRoutes.js ×
backend > routes > JS authRoutes.js > ...
1   const express = require('express');
2   const router = express.Router();
3   const { registerUser, loginUser, getMe } = require('../controllers/authController');
4   const { protect } = require('../middleware/authMiddleware');
5
6   router.post('/register', registerUser);
7   router.post('/login', loginUser);
8   router.get('/me', protect, getMe);
9
10  module.exports = router;
```

requestRoutes.js

```
JS requestRoutes.js ×
backend > routes > JS requestRoutes.js > ...
1   const express = require('express');
2   const router = express.Router();
3   const { protect, staffOnly } = require('../middleware/authMiddleware');
4   const { createRequest, getMyRequests, getAllRequests, updateRequestStatus } = require('../controllers/reque
5
6   // Student routes
7   router.post('/', protect, createRequest);
8   router.get('/myrequests', protect, getMyRequests);
9
10  // Staff only routes
11  router.get('/all', protect, staffOnly, getAllRequests);
12  router.put('/:id/status', protect, staffOnly, updateRequestStatus);
13
14  module.exports = router;
```

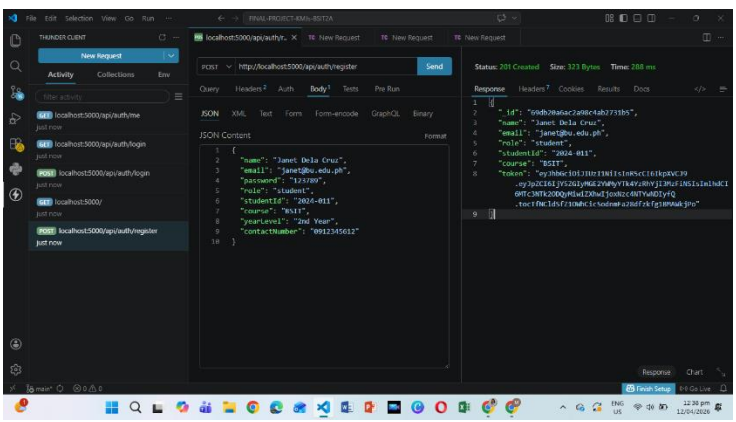
staffRoutes.js

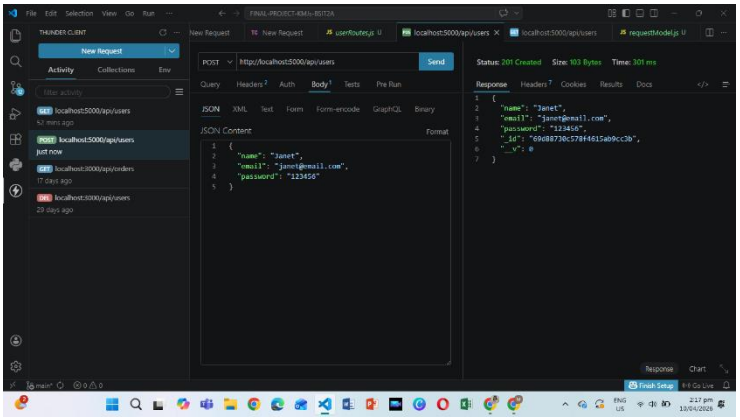
```
JS staffRoutes.js ×
C:\Users\Geline\Desktop\FINAL-PROJECT-KMJs-BSIT2A\backend\routes\staffRoutes.js (preview)
2   const router = express.Router();
3   const { protect, staffOnly } = require('../middleware/authMiddleware');
4   const { getAllStudents, getDashboardStats, updateStudent, deleteStudent } = require('../controllers/staffCo
5
6   router.get('/students', protect, staffOnly, getAllStudents);
7   router.get('/stats', protect, staffOnly, getDashboardStats);
8   router.put('/students/:id', protect, staffOnly, updateStudent);
9   router.delete('/students/:id', protect, staffOnly, deleteStudent);
10
11  module.exports = router;
```

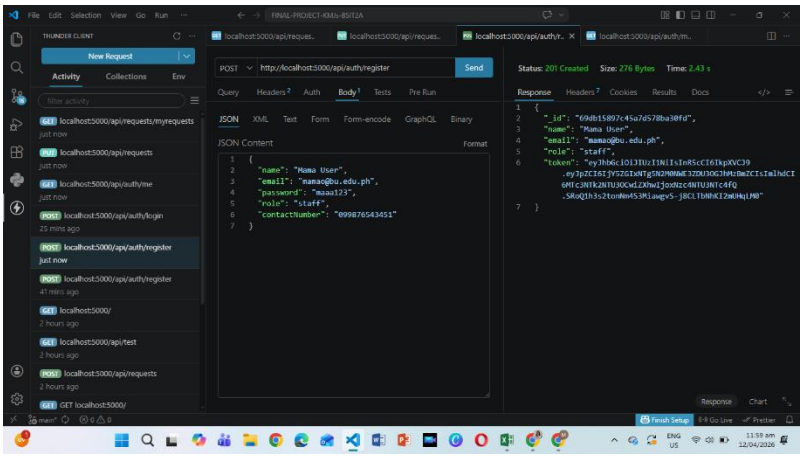
userRoutes.js

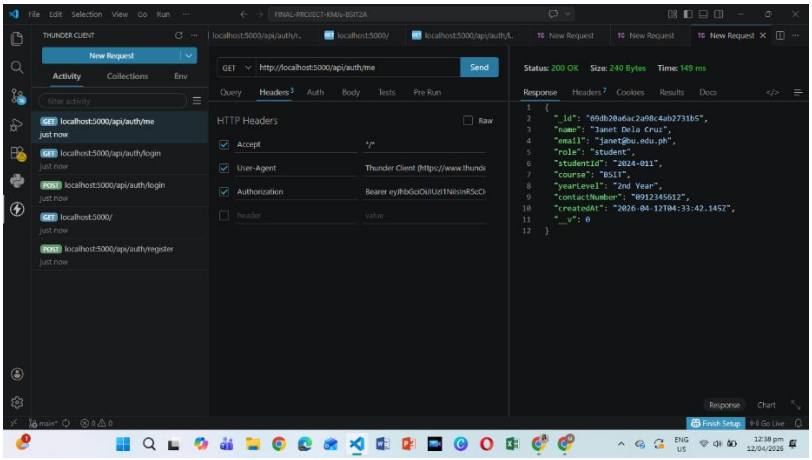
```
JS userRoutes.js ×
backend > routes > JS userRoutes.js > ...
1   const express = require('express');
2   const router = express.Router();
3   const User = require('../models/User');
4   const { protect, staffOnly } = require('../middleware/authMiddleware');
5
6   // Get all users (Staff only)
7   router.get('/', protect, staffOnly, async (req, res) => {
8     const users = await User.find().select('-password');
9     res.json(users);
10  });
11
12  // Get single user
13  router.get('/:id', protect, async (req, res) => {
14    const user = await User.findById(req.params.id).select('-password');
15    res.json(user);
16  });
17
18  // Update user
19  router.put('/:id', protect, async (req, res) => {
20    const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true }).select('-password');
21    res.json(user);
22  });
23
24  // Delete user (Staff only)
25  router.delete('/:id', protect, staffOnly, async (req, res) => {
26    await User.findByIdAndDelete(req.params.id);
27    res.json({ message: 'User deleted' });
28  });
29
30  module.exports = router;
```

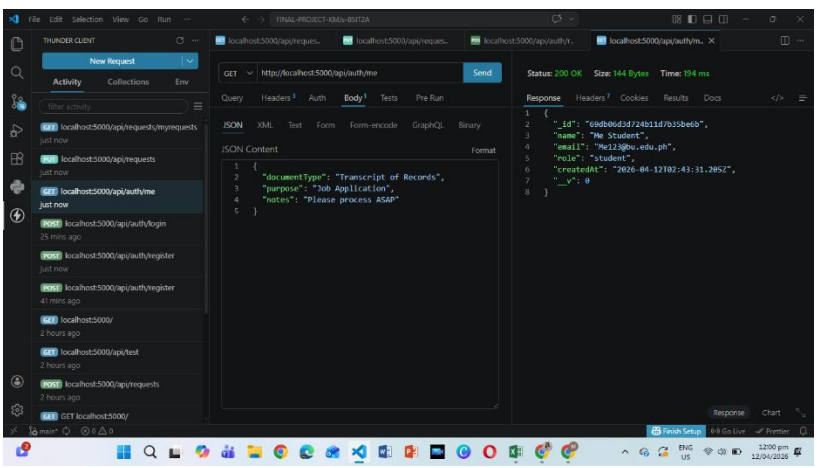
III. API TESTING SCREENSHOTS











IV. BRIEF EXPLANATION OF EACH GROUP MEMBER'S CONTRIBUTION

Github Manager (Janet L. Realubit)

As the Backend Developer, I was responsible for building the complete server-side infrastructure of BU Transakto. I set up the Express.js server, established the folder structure, and configured MongoDB connection using Mongoose. I designed and implemented two database models: User

for student and staff accounts with password hashing, and Request for document submissions with status tracking. I developed the authentication system using JWT tokens, created register and login endpoints, and implemented middleware for role-based access control. I built all CRUD operations including request submission, viewing requests, updating statuses, and user management. I also created staff dashboard endpoints for statistics and student management. I tested all 14 API endpoints using Thunder Client, ensuring proper functionality and error handling. I documented my code with clear comments and collaborated with my team to commit all backend files to GitHub with descriptive commit messages, meeting all Phase 3 requirements for the project.

Front-end Manager (Kimberly Anne Intia)

As a front-end developer, I played an important role in building our system with the help and support of my groupmates, focusing mainly on designing and developing the user interface to ensure it was clean, user-friendly, and well-organized. We collaborated from the planning stage, where we discussed the system layout and interface design, up to the actual development where we turned our ideas into a functional system. My contribution centered on translating our plans into an interactive and responsive front-end, improving the user experience by refining layouts, fixing interface issues, and adjusting the design based on team feedback. This teamwork allowed us to share ideas, solve problems, and enhance the overall quality of the system. Through this experience, I strengthened both my technical skills in front-end development and my ability to work effectively and collaboratively within a team.

Github Manager (Juvy Bellen)

On week 12, where this focused on the phase 3 of our group project which is the back-end setup, my contribution to this phase, since I was assigned as the repository manager I was the one responsible and handling the version control process, especially in committing the back-end updates connected to the repository. This is just to ensured that all the back-end development where it's properly tracked, organized, and also synchronized within our group the KMJ's.

Tester and Debugger (Jesseca Mae Septo)

Working as a tester and debugger of BU Transakto -the web-based system, I tested the web and cooperated with other group members to find out any issues or bugs that might happen in the usability of the website. Tested the basic services including registration, login, document request and dashboard. Some bugs found while testing are “404 Not Found” in Thunder Client and some problem had happened in front-end pages.Together, we analyzed these issues, identified their root causes, and applied appropriate fixes, particularly in API endpoints, routing, and user interface behavior. I actively participated in debugging errors, validating data flow between the frontend and backend, and ensuring that each feature performs as expected. Through teamwork and continuous testing, we were able to improve the system’s reliability, functionality, and overall user experience.

Database Manager (Ma. Mera Hermo)

As the Database Manager for Phase 2, I created a MongoDB Atlas account for our team and set up the cloud database cluster. I configured the connection string in the .env file and whitelisted IP addresses in the Atlas network access panel to allow team members to connect securely. I also assisted in testing the database connection to ensure data persistence for user registrations and document requests.